Command Line Arguments

- Generally main( ) function takes no arguments
- main( ) function can take special arguments and parameters are to be passed to the main from operating system
- Two arguments are passed to the main( ) function

- argc : integer type representing the number of arguments passed
- argv : array of strings, each string representing a parameter passed to main ( )

- Execution of the program is initiated by specifying the name of the program followed by the parameters separated by space
- The program name is interpreted as an operating system command and the line in which it appears is called command line
- As the arguments are provided at command line, they are called command line arguments

Example program 1: Command line arguments

```
#include <stdio.h>
 int main( int argc, char * argv[ ])
  {
     int i;
       for (i = 0; i< argc; i++)
        printf ("%s \n", argv[i]);
       return 0;
      }
```

- At the command line, the following command can be given
- C:\> *claprog first second third*
- argv[0] is the program name *claprog*, argv[1] is first argument *first*,
- Argv[2] is second argument *second* and argv[3] is third argument *third*

Example program 2: Command line arguments

- In this example, we open a file and copy the contents to another file. The name of the two files will be taken as input from the user at command line
- Let us name the program as copyfile.c

```
#include <stdio.h>
 #include<stdlib.h>
 int main (int argc, char *argv[])
 //int argc; char *argv[];
    {
      FILE *fs, *ft;
      char ch;
      if ( argc != 3)
      //{  printf (“\n Insufficient arguments”);
      { puts(" Insufficient arguments\n");

        exit(1);
      }
      fs = fopen (argv[1], "r");
      //fs = fopen (argv[1], “r”);
    if ( fs == NULL)
    {
                //puts (“Source file could not be opened”);
                puts ("Source file could not be opened");
   exit (2);
}
```

Example program 2: Command line arguments contd..

```
ft = fopen(argv[2], "w");
 if (ft == NULL)
 {
 puts ("can not open target file");
  fclose(fs);
 exit ( 3);
 }
```

Example program 2: Command line arguments contd..

```
while (1)
 {
   ch =fgetc (fs);
    if (ch == EOF)
  break;
else
   fputc (ch, ft);
}
fclose(fs);
fclose (ft);
return 0;
}
}
```

C:\> copyfile source destination

source and destination will have the same contents

Enumerated Data Type

- Enumeration (or enum) is a user defined data type in C
- It is mainly used to assign names to integral constants
- The names make a program easy to read and maintain.
- The keyword ‘enum’ is used to declare enumeration types in C
- Following is an example of enum declaration
- enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
- enum week day;
- week is an enumerated data type; any variable of this type can take only the enumeration constant values mentioned in the declaration
- day is a variable of enum type week; so day can take one of the week days
- enum weekend { Saturday, Sunday};
- enum weekend holiday;
- holiday = Saturday
- The compiler automatically assigns integer digits beginning with 0 to all the enumeration constants
- For example, Mon = 0, Tue = 1, Wed = 2 and so on.

Example: Program to print day of the week

- /* An example program to demonstrate working of enum in C*/
- #include<stdio.h>
- enum week{Mon, Tue, Wed, Thur, Fri, Sat, Sun};
- int main()
- {
- enum week day;
- day = Wed;
- printf("%d",day);
- return 0;
- }
- Output:
- 2
- In the above example, we declared “day” as the variable and the value of “Wed” is allocated to day, which is 2; as a result, 2 is printed.
- Exercise: Write a program to check a day in a week is working day or a holiday and print accordingly “Working day” or “Holiday”

Using GCC for programming in C on Linux Platform

- Let us know how to Write and Run a C Program in Linux
- Linux is an open-source and free operating system.
- Let us explore to write, compile, and run a simple C program
- This will serve as a basis for you to move to more complicated and useful C programs that you can write and execute on Linux
- You can use Ubuntu or any Linux operating system
- To open the Terminal, you can use the Ubuntu Dash or the Ctrl+Alt+T shortcut
- Step1: In order to compile and execute a C program, you need to have the essential packages installed on your system
- Step 2: Write a simple C program
- After installing the essential packages, let us write a simple C program.

- Open Ubuntu's graphical Text Editor and write or copy the following sample program into it

Example program to run on Linux

#include<stdio.h>

- int main()
- {
- printf("\n A sample C program\n\n");
- return 0;
- }
- Then save the file with .c extension.
- Let us name it *sampleProgram.c*
- Alternatively, you can write the C program through the Terminal in gedit as follows:
- $ gedit sampleProgram.c
- This will create a .c file where you can write and save a program.

Compile and run the example program with gcc compiler

- Step 3: Compile the C program with gcc Compiler
- In your Terminal, enter the following command in order to make an executable version of the program you have written:
- Syntax:
- $ gcc programName.c -o programName
- Example:
- $ gcc sampleProgram.c -o sampleProgram
- Compiles source code with gcc to create an object file named sampleProgram
- Make sure your program is located in your Home folder; otherwise, you will need to specify appropriate path in this command
- Step 4: Run the program
- The final step is to run the compiled C program; use the following syntax to do so:
- $ ./programName
- Example:
- $ ./sampleProgram
- The desired output will be displayed

Functions with Variable number of Argument in C

- Variable length arguments is a feature that allows a function to receive any number of arguments.
- printf is an example of a function with variable number of arguments
- Example problems where they are useful
- 1) Sum of given numbers 2) Minimum of given numbers
- Variable number of arguments are represented by three dots (…) or ellipsis
- The include file <stdarg.h> is to be used for the three macros va_list, va_start and va_arg that are useful to deal with variable arguments

va_start is used to initialize a pointer to the beginning of the list of optional arguments

va_arg is used to advance the pointer to the next argument

va_list holds the information about the arguments

Differences between Macros and functions

- A macro is a list of code with a name
- Whenever the name is used, it is replaced by the code

Macros	**Functions**
• Before compilation, macro name is replaced with a list of code	During function call, transfer of control takes place
• Macros are useful when small code is to be repeated many times	Functions are useful when large code is to be written
• The macros are expanded by the preprocessor	The inline functions are parsed by the compiler; inline function can access class members
Macros are efficient and faster as the code is directly replaced	There is overhead of taking control from function call to called function

Example: Find minimum of a set of numbers

```
/*C program to demonstrate the use of variable number of arguments*/
#include <stdio.h>
/* this function returns minimum of integer numbers passed*/
  /*First argument is count of optional arguments*/
int min(int arg_count, ...)
{
    int i;
    int minm, a;
    /* va_list is a type to hold information about variable arguments*/
    va_list ap;
 /* va_start must be called before accessing variable argument list */
    va_start(ap, arg_count);
/* arguments can be accessed one by one using va_arg macro*/
```

Example: Find minimum of a set of numbers contd..

```
/*Initialize min as first argument in the list */
    minm = va_arg(ap, int);
    printf("initial minm is %d\n",minm);
/* Check rest of the arguments to find out minimum*/
    for (i = 2; i <= arg_count; i++) {
      a = va_arg(ap, int);
      printf("Next integer moved to is %d\n",a);
        if ((a < minm))
            minm = a; }
 /* va_end should be executed before the function returns whenever va_start has been used*/
va_end(ap);
    return minm;
}
```

Example: Find minimum of a set of numbers contd..

```
int main()
{
    int count1 = 5;
     int count2= 6;
    printf("Minimum value is %d\n", min(count1, 11, 7, 16, 17, 100));
  printf("Minimum value is %d", min(count2, 120, 134, 32, 67, 98,200));
    return 0;
}
```

Output

```
Minimum value is 7
Minimum value is 32
```